

Tellix 1.0.0 — Release Notes

Release date: 13 May 2026 **Platform:** Windows 10 / 11 (x64) **Distribution:** Single-file portable executable, ~250 MB

Summary

Tellix is a lightweight Windows screen-recording app with built-in speech-to-text. It records the screen plus microphone (and optionally any audio playing through the speakers, for capturing the other side of a video call), then produces a single MP4 file with subtitles embedded, alongside a timestamped transcript. Everything runs locally — there's no cloud upload and no telemetry.

This is the first public build. The feature set below ships in 1.0.0.

Features

Tellix captures the primary display at 30 fps using FFmpeg's `gdigrab`, with fragmented MP4 flags so the file remains playable even if the recorder is killed abruptly. Microphone capture goes through `sounddevice` (PortAudio); the device is opened at its native sample rate and Tellix resamples to 16 kHz mono in the callback, with a seven-step fallback chain that survives the format-negotiation quirks of MME, DirectSound, and some corporate-managed hardware.

System audio capture — used for recording calls, meetings, or anything where the other side's audio comes through your speakers — is implemented via WASAPI loopback through the `soundcard` library, enabled with a single checkbox in the UI. When system audio is enabled, the mic and loopback streams are mixed sample-by-sample in real time into `mixed.wav` and fed to the transcriber as a single combined stream.

Transcription uses `faster-whisper`, the CTranslate2 reimplementation of OpenAI Whisper. A live pass runs in a background thread against rolling 5-second windows with 1-second overlap so users see partial captions during the recording. The final pass, triggered when the user stops recording, runs against the full audio with a larger model, beam search, language auto-detection, and Whisper's internal silence detection. Output lands as both an SRT subtitle file and a plain text transcript, with the SRT also muxed into the final MP4 as a soft subtitle track (`mov_text` codec) so the captions travel inside the file rather than requiring a sidecar.

A small companion script, `retranscribe.py`, re-runs the final pass against any existing session folder or WAV file — useful for recovering transcripts when settings need to be adjusted (different model size, explicit language, etc.) without re-recording.

Distribution

A PyInstaller spec (`tellix.spec`) and a PowerShell build helper (`build.ps1`) produce a single-file `Tellix.exe` of roughly 250 MB. `app.py` is bundle-aware via `sys._MEIPASS`, so the same code paths work whether you're running from source or from the packaged executable. Recordings are written to

output/ next to the .exe, not into the temp extraction directory, so they persist across runs.

The Whisper model files are intentionally not bundled — they download to %USERPROFILE%\cache\huggingface\ on first transcription and are cached locally thereafter. This keeps the installer small (250 MB versus 750 MB+ if the small model were bundled).

What's inside the .exe

The bundle includes ffmpeg.exe (~80 MB static Windows build), the Python interpreter and Tkinter runtime, all native DLLs needed by the audio stack (PortAudio for sounddevice, CTranslate2 for faster-whisper, the C++ runtime), and the pure-Python dependencies: faster-whisper, ctranslate2, tokenizers, sounddevice, soundcard, numpy. Heavyweight libraries that aren't used (torch, tensorflow, matplotlib, pandas, scipy, PyQt) are excluded explicitly in the spec to keep the binary small.

System requirements

Windows 10 or 11 on x64. 4 GB RAM minimum, 8 GB recommended if you plan to use the small or medium Whisper models. About 500 MB of free disk for the Whisper model cache, plus enough room for your recordings (roughly 10–15 MB per minute of 1080p screen capture). NVIDIA GPU is optional — when present, transcription runs roughly 5× faster via CUDA. The application falls back to CPU automatically when no GPU is detected.

Distribution notes

The produced Tellix.exe is **unsigned**. On first launch on a new machine, Microsoft Defender SmartScreen shows a "Windows protected your PC" dialog. Users click "More info" and then "Run anyway" to proceed. After that one click, Windows trusts that specific .exe on that machine and doesn't prompt again. For real public distribution this is solved with code-signing — an EV (Extended Validation) certificate eliminates the warning instantly; standard OV certificates build reputation over time. Either is worth the cost (~\$80–500 per year) if Tellix is being distributed at scale.

The .exe is portable. Drop it in any folder on any Windows 10/11 machine and it works. Required network access is limited to the one-time Whisper model download from HuggingFace, which happens transparently the first time a recording is transcribed; subsequent recordings are fully offline.

Known limitations

Live captions in the on-screen pane lag the speaker by 5–7 seconds because of the rolling-window size. This is expected; the final transcript that lands in recording.txt is exact and uses a larger model. VAD filtering is enabled on the live path (to suppress hallucinations during silences) but disabled on the final pass (because Silero VAD at default settings dropped quiet speech entirely). The trade-off on the final pass is that very long silences may occasionally produce short hallucinated phrases; in practice these are rare and easy to spot.

Pause/resume during a recording is not supported. FFmpeg can't pause cleanly, so a pause feature would have to record in segments and concatenate at stop time. Screen capture targets the primary display only

— multi-monitor setups capture the primary monitor's region; selecting a different monitor or a sub-region would require exposing FFmpeg's `-offset_x`, `-offset_y`, and `-video_size` options in the GUI. The Tellix window itself appears in screen recordings because `gdigrab` captures everything; a global hotkey to start/stop without focusing the window would solve this and is on the roadmap.

Development changelog

The notable changes during development, in order of when they shipped:

The initial walking skeleton wired FFmpeg screen capture, sounddevice mic capture, and faster-whisper into a Tkinter GUI without system audio or subtitle muxing.

The architecture was then tightened so sounddevice owns the microphone exclusively. The obvious design — letting FFmpeg grab both screen and mic in one command — would have prevented live transcription on most Windows machines, because DirectShow rarely permits two clients on the same mic.

VAD filtering was added on the live transcription path to stop Whisper from hallucinating text during silences in the rolling 5-second windows.

System-audio capture was added via WASAPI loopback. The first attempt used sounddevice's `wasapiSettings(loopback=True)`, which turned out to never have been a real API in any released sounddevice version. The implementation switched to the `soundcard` library, which exposes `include_loopback=True` explicitly. A real-time mixer thread was added to combine mic and loopback into a single 16 kHz mono stream for the transcriber.

A transcription regression was diagnosed and fixed: forced `language="en"` plus aggressive Silero VAD on the final pass was dropping users' speech entirely. The fix was language auto-detection plus disabling VAD on the final pass. A `retranscribe.py` helper was added so existing recordings with bad transcripts could be recovered without re-recording.

`AudioCapture` was rewritten to open the mic at its native sample rate and resample in the callback after a user hit a PortAudio MME `-9999 "Undefined external error"` on a different laptop's Intel Smart Sound mic. The new fallback chain tries up to seven format and host-API combinations before giving up.

The finalize sequence was reordered to transcribe first, then mux video + audio + `.srt` with the `mov_text` subtitle codec — so captions travel inside `recording.mp4` rather than as a sidecar `.srt`. This fixed a real complaint where copying just the `.mp4` to another machine lost the captions.

`PyInstaller tellix.spec` and the `build.ps1` build helper were added, with `app.py` made bundle-aware via `sys._MEIPASS` so recordings land next to the `.exe` rather than in the temp extraction directory.

Roadmap

Likely candidates for the 1.1 release: a "Burn captions into video" option for users who want the captions baked into the pixels rather than as a toggleable track; a global hotkey for Start and Stop so the Tellix window can be hidden from the recording itself; multi-monitor and region capture; speaker diarization in

the transcript (would require `pyannote.audio` or similar — Whisper alone can't do this); a configurable framerate slider; an explicit retranscribe button inside the GUI so users don't need to drop to the terminal.

Acknowledgments

Tellix is built on FFmpeg, OpenAI Whisper (via `faster-whisper` and `CTranslate2`), `sounddevice` and `PortAudio`, the `soundcard` library, `NumPy`, and Python's standard library `Tkinter`. Thanks to the maintainers of all of those projects.